Plug-in CaWaQS-Viz 0.1.17

Visualizer for the modelling platform CaWaQS 3.55

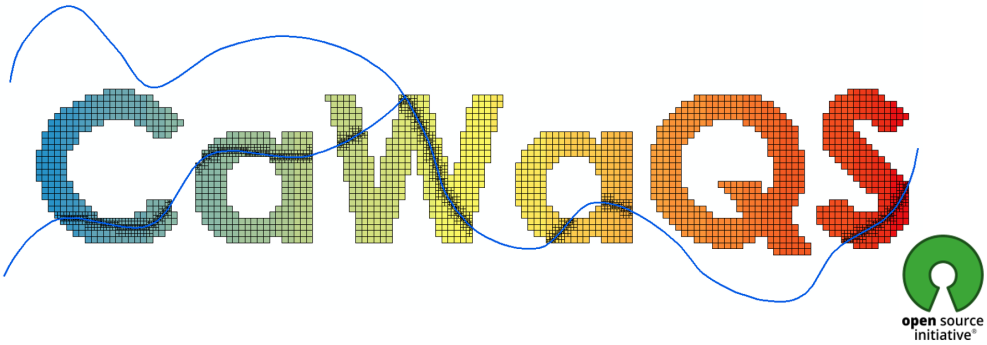
User Guide

Lise-Marie GIROD

Simone MAZZARELLI

Nicolas GALLOIS

Nicolas FLIPO



March 23, 2026

Mines Paris/ARMINES, PSL Université - Centre de Géosciences
35, rue Saint-Honoré
F-77 305 FONTAINEBLEAU Cedex

Introduction

The visualization tool CaWaQS-Viz was developed with the aim of visualizing, within a GIS environment, the modelling outputs of the CaWaQS 3.55 modelling platform. This extension currently operates under QGIS version 3.34.

CaWaQS-Viz is structured around various tools and features that assist in the calibration process of a CaWaQS application and facilitate the visualization of time series and the spatial distribution of simulated hydrological variables.

What's New

Below is the list of the latest bug fixes and feature implementations in the CaWaQS-Viz application.

Type/Tag	Description	Section
New feature	Integration of an aquifer mesh where the column identifiers containing the absolute cell identifiers differ between shapefiles	Sec. **??**
New feature	Application parameterisation with extraction points that have no observation data	Sec. **??**
New feature	Application configuration without an observation data directory	Sec. **??**
New feature	Calculation of global and per-aquifer-unit statistical criteria	Sect. **??**

Table 1: Latest feature implementations in the application

Latest releases:

CaWaQS-Viz 0.1.14 : **Release** (20 March 2025) : First stable release of CaWaQS-Viz
CaWaQS-Viz 0.1.16 : **Beta Release** (30 July 2025) : development release

Contents

List of Figures

List of Tables

List of Codes

CHAPTER 1

Prerequisites and Installation

Contents

The CaWaQS-Viz plug-in was designed to run within the QGIS Geographic Information System (GIS). It was initially developed under QGIS 3.28[1], and is currently used under version 3.36. Correct operation is not guaranteed for use under later versions.

1.1 Dependency Installation

In its first version, CaWaQS-Viz does not yet provide a dependency installer. It will therefore be necessary to install dependencies before installing CaWaQS-Viz.

1.1.1 Installation on Microsoft Windows

Installing dependencies on Microsoft Windows is most easily done directly using the Python interpreter integrated into QGIS. To do so, open a Python console[2] and type the following commands:

```
>>> import pip
>>> pip.main(['install', 'pandas', 'numpy', 'ipython', 'matplotlib', 'geopandas'])
```

Installing dependencies from a Python interpreter other than the one used by your version of QGIS will not allow QGIS to access the libraries required by the plug-in. Make sure to install the necessary dependencies from the Python interpreter built into QGIS.

1.1.2 Installation on Linux

Installing dependencies on a Linux distribution is done from the command terminal. Make sure to install the following dependencies: pandas, numpy, ipython, matplotlib, geopandas.

1.2 Standard Plugin Installation (Users)

The plug-in is installed *via* QGIS, by importing the plug-in archive (`.zip`), previously downloaded from the GitLab repository (https://gitlab.com/gviz/cawaqsviz) of the extension (**??**). Once downloaded, open QGIS.

[1] Available at: https://www.qgis.org/fr/site/forusers/download.html

[2] By clicking on  in the QGIS interface.

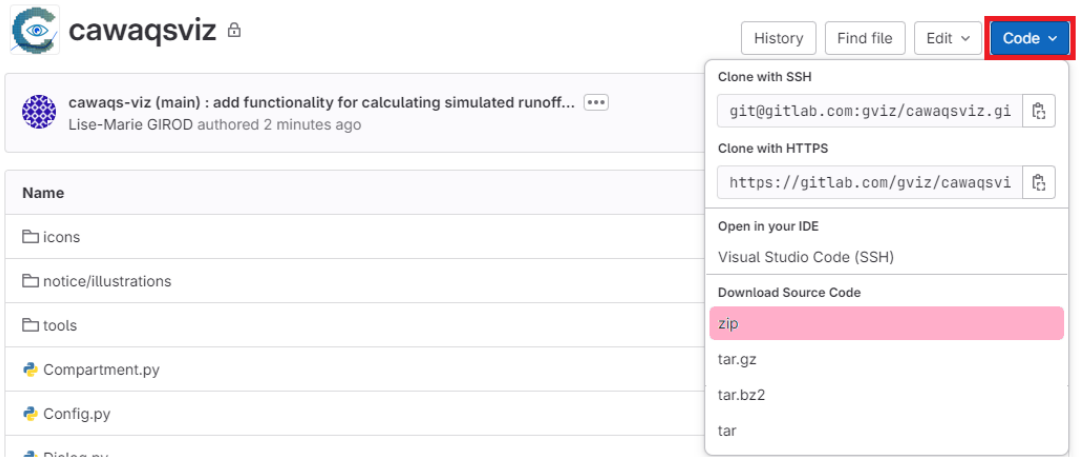


Figure 1.1: Downloading the CaWaQS-Viz archive from the GitLab repository web interface.

In the QGIS top menu bar, click on "`Plugins`" then "`Manage and Install Plugins`". The plugin management and import window opens. Select "`Install from ZIP`" (**??**).

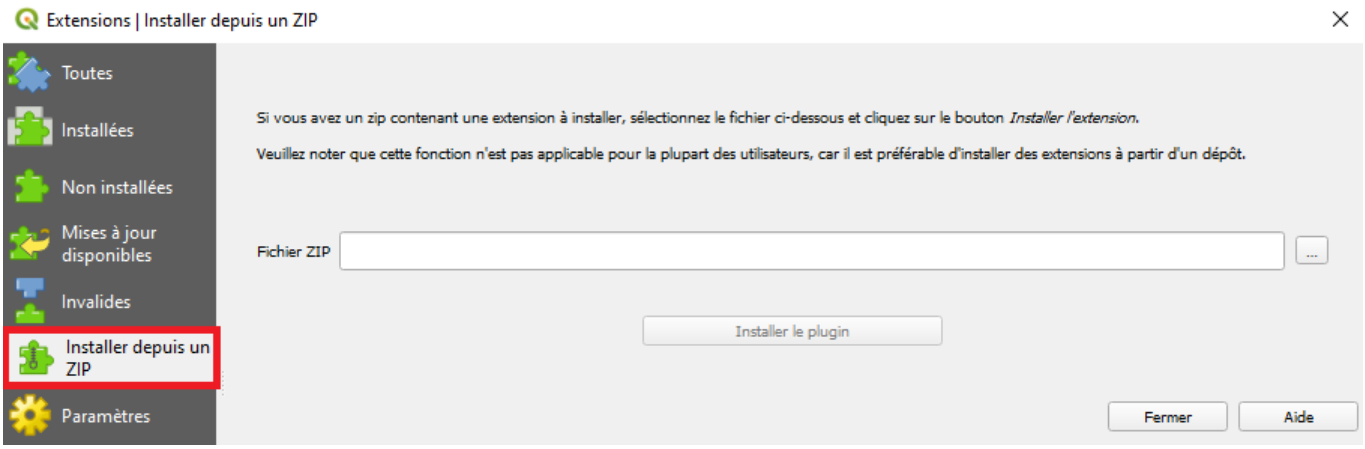


Figure 1.2: QGIS dialog box for plugin management.

Once the `.zip` file has been extracted in QGIS, it may be necessary to activate it in the same plugin management window by navigating to the "`Installed`" panel. The CaWaQS-Viz extension must be checked to appear in the QGIS toolbar (**??**).

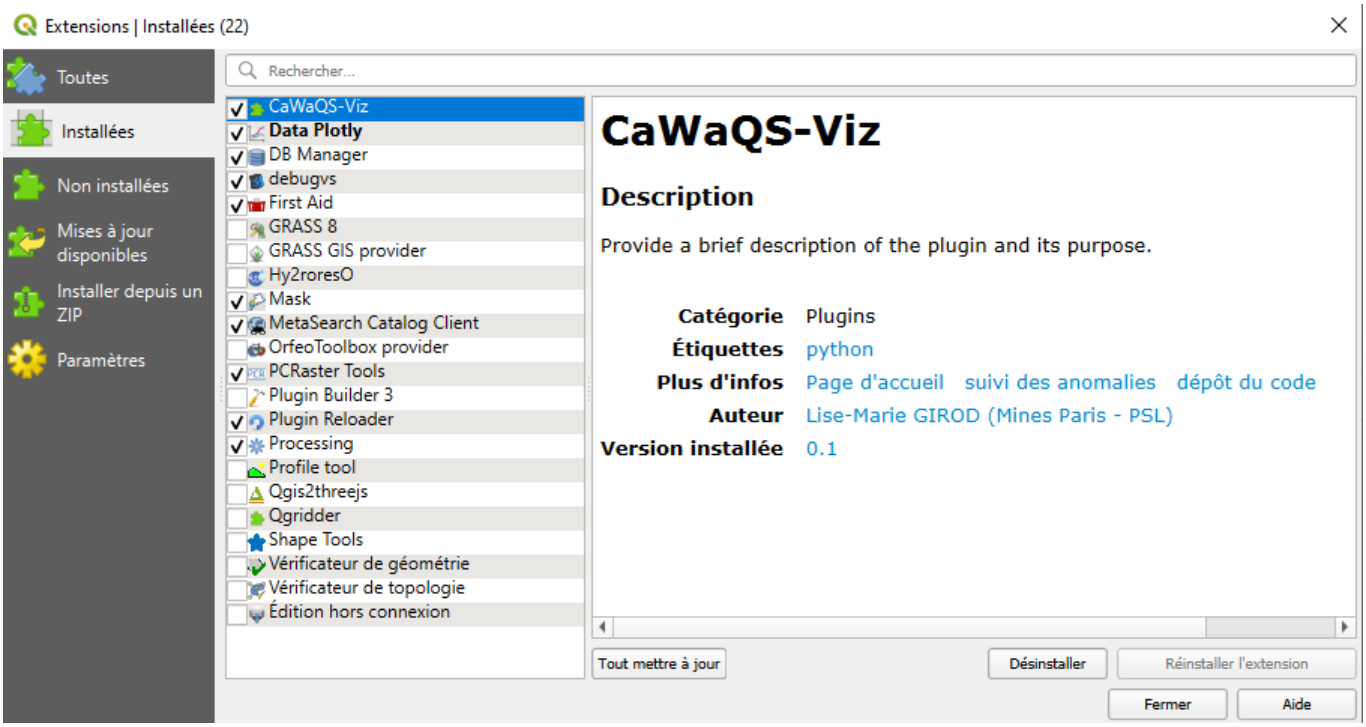


Figure 1.3: Activating the CaWaQS-Viz extension in the QGIS application.

Once the application is present in the QGIS toolbar, it is ready to be launched with a single click on the icon .

> To use any new update of the plug-in in QGIS, it will be necessary to repeat this entire procedure (**??**).

1.3 Developer Installation

The GitLab repository for CaWaQS-Viz can be cloned from a command terminal:

```
$ git clone https://gitlab.com/gviz/cawaqsviz
```

The application directory must be placed at the location (folder) of the native QGIS applications so that any modification of the plug-in source code can be taken into account by QGIS at startup. The QGIS directory is accessible *via* the following path:

```
#windows
C:/Programmes/QGIS3.X.X.X/apps/qgis-ltr/python/plugin/
#linux
/home/bin/QGIS3.X.X.X/apps/qgis-ltr/python/plugin/
```

The plug-in code structure is described in chapter **??**.

Autocompletion of features related to the qgis library is available provided the Python interpreter used by QGIS is used. A virtual environment can be initialised for this purpose.

N.B.: To facilitate plug-in development, the following utilities can be installed from the QGIS application manager:

1. **Reloader** allows reloading the plug-in without restarting QGIS after each code modification,
2. **FirstAid** provides a detailed view of errors returned by the QGIS Python interpreter as well as the various objects and variables related to the extension.

CHAPTER 2

Features and Configuration of the CaWaQS-Viz Extension

Contents

2.1 Overview

The CaWaQS-Viz extension integrates several features accessible from its home dialog box (**??**):

- a dedicated entry for configuring the CaWaQS application ("`Settings`" - **??** to **??**) ;
- calibration assistance tools ("`Calibration tools`" - **??**) ;
- features for visualizing simulated and observed flow and piezometry time series ("`Compare Sim/Obs`" - **??**) ;
- tools for visualizing the spatial distribution of hydrological components ("`Spatial Maps`" - **??**) ;
- a hydrological budget visualization function ("`Budget Bar Plot`" - **??**) ;
- a hydrological regime viewer ("`Hydrological regime`" - **??**).

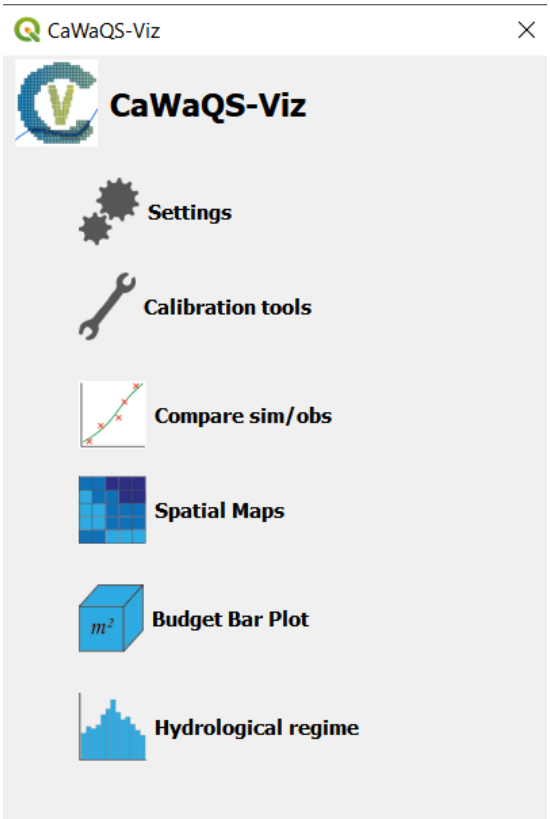


Figure 2.1: Home dialog box of the CaWaQS-Viz application.

2.2 Plugin CaWaQS-Viz Configuration

The use of the CaWaQS-Viz extension is based on a pre-existing `GIS` project containing:

- the meshes of the various CaWaQS application modules, at the modelling output resolutions;
- observation points and/or extraction points, located within the bounds of the model extent.

The plug-in must be configured to read the various modelling output files and/or CaWaQS input files as well as the various meshes. This parameterisation of the plug-in can be carried out in the "`Settings`" window. To do so, two procedures are available:

- import a `.json` configuration file established beforehand;
- configure the project manually in the second part of the dialog box shown in figure **??**.

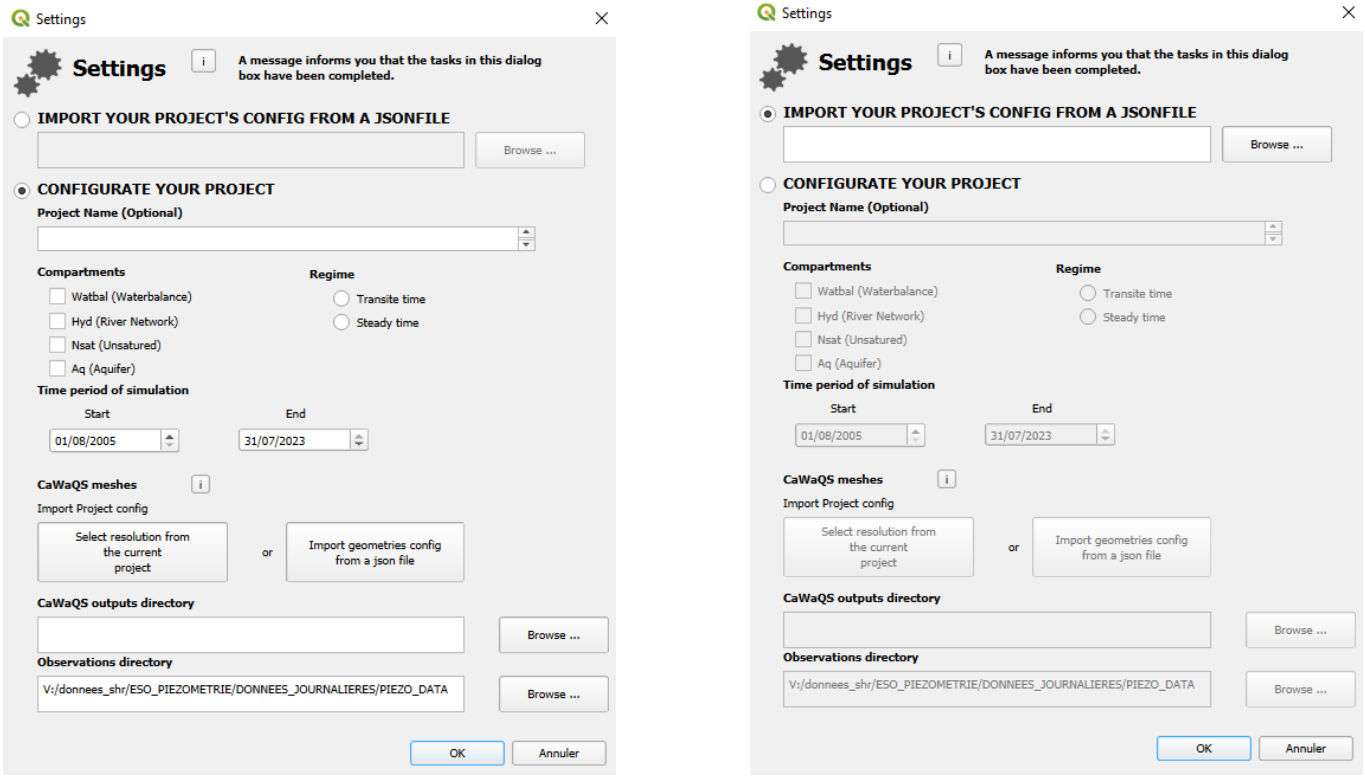


Figure 2.2: The "`Settings`" dialog box of the plug-in. The CaWaQS-Viz application configuration method must be selected: **(a)** creation of a new project **(b)** importing a project configuration.

The details of these two application configuration procedures are described in paragraphs **??** and **??** respectively.

2.3 Manual Plugin Configuration: First Use

The lower part of the "`Settings`" dialog box allows manual configuration of the plug-in. The following elements must be defined:

1. a project name;
2. the modelled compartments;
3. a GIS layer mesh configuration;
4. the modelling output and observation directories[1] if observation points are defined in the project.

[1]The flow and piezometry observation directories are assumed to be in the same parent directory.

The project name will be used to name a sub-folder created in the post-processing directory. The graphical outputs of the CaWaQS-Viz application will be placed in this post-processing folder.

The geometry configuration can also be imported from a `.json` configuration file (by clicking on "**`Import a geometry configuration from a .json`**") or defined manually *via* the option "**`Select resolutions from the current project`**".

Note that any manual project and geometry configuration will be exported in `.json` format to the post-processing directory upon execution of the "**`Settings`**" dialog box.

2.3.1 Contents of the QGIS Project Used for Processing CaWaQS Simulation Results

The `shapefiles` of the CaWaQS model meshes

The QGIS project used for processing simulations with the CaWaQS-Viz utility must contain:

- the meshes (resolutions) of the compartments in "**`shapefile`**" format of the model (refer to the CaWaQS documentation[2] to identify the output resolutions of the "**`WATBAL`**" compartment). The CaWaQS resolution types to import into the GIS project for processing modelling outputs depend on the compartments actually being modelled;
- the observation points of the compartments to be defined in the CaWaQS-Viz extension.

	Surface compartment (WATBAL)	**Hydraulic compartment (HYD)**	**Unsaturated compartment (NSAT)**	**Aquifer compartment (AQ)**
CaWaQS resolution	Unit surface cell (`ele_bu`)[3] or Production cell (`Cprod`)	River element (`ele_musk`)	Unsaturated zone grid	Meshes of all aquifer layers
Required fields in the attribute table	Cell identifiers	GIS identifiers of river elements	Cell identifiers	Absolute cell identifiers

Table 2.1: CaWaQS resolutions that can be imported as compartment resolution.

The names of the GIS layers in the project corresponding to aquifers must be identical to those defined in the CaWaQS configuration to ensure correct application behaviour.

The `shapefiles` of extraction points

[2]CaWaQS geometry terminology. For more information on the different types of meshes, refer to the model documentation (*i.e.* user guide and technical guide), available on the GitLab repository https://gitlab.com/cawaqs/cawaqs.

Extraction of simulated flow and hydraulic head can be performed at extraction points. Two types of extraction points can be defined:

- simple extraction points, which do not include a measurement time series;
- observation points, which are extraction points with measurement data.

Extraction and observation points are contained in two separate `shapefile` files whose attribute tables contain the fields listed in table **??**.

POINT TYPE	ATTRIBUTE TABLE CONTENTS	OUTPUTS
Extraction point	• Point name • Identifier of the attached cell[4], aquifer layer identifier (if extraction in aquifer)	• Simulated hydrographs and piezometric curves at the extraction point
Observation point	• Point name • Identifier of the attached cell [4]	• Simulated and observed hydrographs and piezometric curves • Objective criteria comparing observations with simulations

Table 2.2: Structure of GIS layers for extraction points and associated graphical outputs

Observation `shapefiles` must be imported into the project and specified in the CaWaQS-Viz configuration if the HYD and/or AQ compartments are being processed. Without observations, the extension will not be functional.

2.3.2 Configuring the Project Geometry from an Existing QGIS Project

The geometric configuration of the CaWaQS-Viz application can be defined from an existing QGIS project via the "`Application configuration from an existing project`" dialog box in the application settings dialog.

The window is organised into two sections:

- on the left, the CaWaQS compartments are listed (WATBAL, HYD, AQ, NSAT),
- on the right, the parameters to be defined are listed for producing graphical outputs (flow and piezometry) at observation points (extraction points with observations) or extraction points (without observations).

If the application configuration fields are left empty, the compartment concerned is not included in the geometric configuration.

The compartment fields are configured as follows:

The surface compartment (""WATBAL COMPARTMENT"") :

- "Surface resolution": output resolution of the surface water balance (CPROD or elebu);
- "cell id": the column number in the attribute table corresponding to the cell identifier;
- "cell area": the column number in the attribute table corresponding to the cell area (optional parameter).

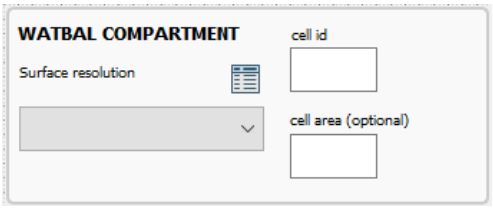


Figure 2.3: WATBAL compartment configuration fields

The hydraulic compartment ("HYD COMPARTMENT")

- "Hyd resolution": name of the output resolution of the "HYD" compartment (river elements);
- "GIS Id": the column number in the attribute table corresponding to the GIS identifier of the cells

Figure 2.4: HYD compartment configuration fields

The unsaturated compartment ("NSAT COMPARTMENT"):

- "Unsatured zone resolution": name of the output resolution of the "NSAT" compartment;
- "cell id": the column number in the attribute table corresponding to the cell identifier.

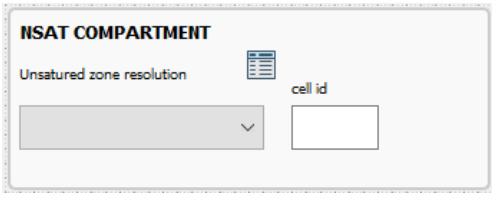


Figure 2.5: NSAT compartment configuration fields

The saturated compartment ("AQ COMPARTMENT")

- "`Aquiferes resolutions`": the GIS layers constituting the underground mesh. The selected layer is added by clicking on "`Add`". Layers must be ordered from the uppermost to the lowermost layer using the "`up`" and "`down`" buttons. Finally, a layer can be removed from the list by clicking on "`remove`";
- "`cell id abs`": identifier of the attribute table column containing the absolute cell identifier. This identifier can be homogeneous across GIS layers: in that case, "`homogeneous ID ABS`" must be selected. Otherwise, click on "`heterogeneous ID ABS`" to open dialog boxes allowing you to specify, for each layer, the column containing the ABS ID of the cells (cf. Fig **??**).

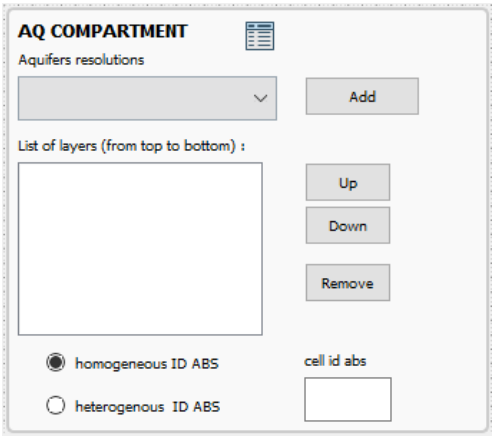


Figure 2.6: AQ compartment configuration fields

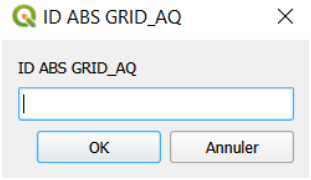


Figure 2.7: Window for selecting the field identifier containing the "`ID ABS`" in the attribute table in the case of heterogeneous field identifiers across layers

Some points of caution:

- **The identifier of the first column of an attribute table starts at 0**. The icon allows viewing the identifiers of the fields listed in the attribute table of the resolution selected in the drop-down list;
- the code (e.g., BSS code) of an observation point must be **unique** among all observation points;
- The geometric configuration of the compartments checked in the project parameterisation dialog box must be defined in the geometry configuration dialog box.

Extraction points - "`Hydrological and Hydrological plots`"

Extraction points can be of two types:

- simple extraction points, which allow viewing simulated variables at those points;
- observation points, which allow comparing simulated variables with observation data at those points.

The configuration of extraction points requires specifying in the "`GIS Layer`" fields the name of the layer in the QGIS project containing the observation points and extraction points according to the relevant `CaWaQS` compartment. The field identifiers to be specified are as follows:

Extraction points of the "HYD" compartment

Extraction points

- "name": name of the observation point;
- "ID GIS": GIS identifier of the observation point to which the observation point must be attached (optional field)

Observation points

- "code": identification code of the observation point, must be unique for each point. The observation file for the point must have the same name with the .dat extension;
- the "name" and "ID GIS" fields are also specified for the configuration of observation points

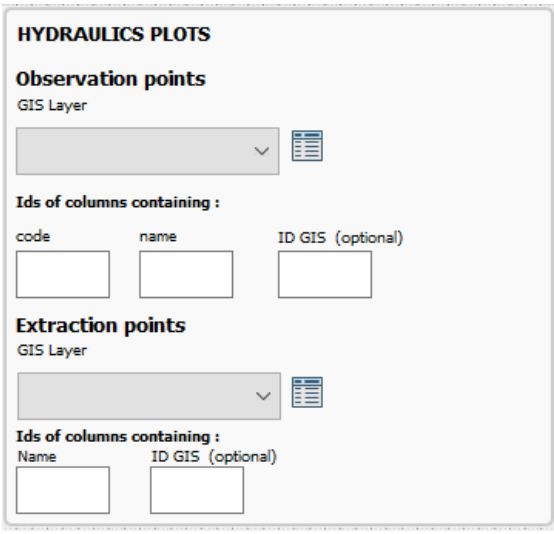


Figure 2.8: Configuration fields for "HYD" compartment extraction points

Extraction points of the "AQ" compartment

Extraction points

- "name": name of the observation point;
- "ID LAYER": identifier of the aquifer layer
- "ID GIS": GIS identifier of the observation point to which the observation point must be attached (optional field)

Observation points

- "code": identification code of the observation point, must be unique for each point. The observation file for the point must have the same name with the .dat extension;
- the "name" and "ID GIS" fields are also specified for the configuration of observation points

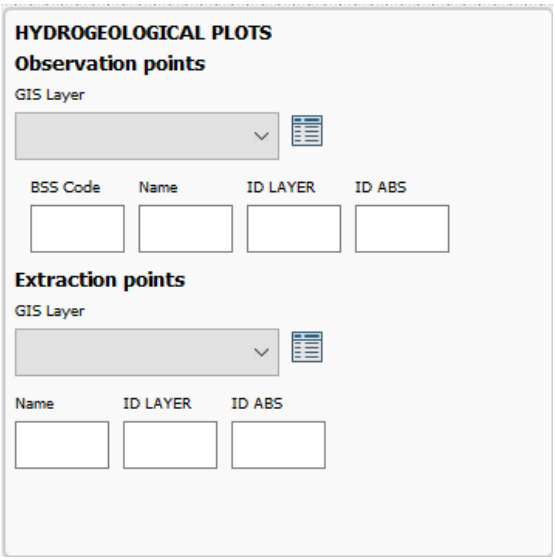


Figure 2.9: Configuration fields for "AQ" compartment extraction points

Extraction points must be contained in a .shp file different from the observation points. Points cannot be contained in the same file if they are extracted from two different compartments.

A preliminary save of the geometry configuration file can be saved by specifying a file path in the field located at the bottom left of the dialog box. This file (cf. Code **??**) will be saved upon execution of the "**Settings**" dialog box if the configured project does not yet exist in the post-processing directory specified. Multiple project configurations can be saved in the same post-processing directory, provided they do not have the same project name.

2.3.3 Configuring Resolutions from a `.json` Configuration File

The geometric configuration of resolutions can be specified via a `.json` file according to the following structure:

```
{   // list of CaWaQS compartments to analyse (1 = AQ, 2 = HYD, 3 = WATBAL)
    "ids_compartment": [],

    "resolutionNames":
    {
        "1":
            [[""]], // list of names of aquifer meshes in the QGIS project
        "2" :
            [[""]], // name of the river mesh in the QGIS project
        "3" :
            [[""]] // name of the surface mesh in the QGIS project
        "4" :
            [[""]] // name of the unsaturated zone mesh in the QGIS project
        },
    "ids_col_cell": { // "compartment id" : "column number of cell id"
        "1" : "",  // "AQ id" : "absolute id column number"
        "2" : "",  //"HYD id" : "GIS id column number"
        "3" : ""   // "WATBAL id" : "cell id column number"
        },
    "obsNames": {
    // "compartment id" : "name of associated obs shp in the QGIS project"
        "1": "",
        "2" : ""
        },
    "obsIdsColCells": {
    // "compartment id" : "field number of the measurement point code"
        "1": "",
        "2": ""
        },
    "obsIdsColNames": {
    // "compartment id" : "field number of the measurement point name"
        "1": "",
        "2" : ""
        },
    "obsIdsColLayers": {
    // "compartment id" : "field number of the aquifer layer identifier"
```

```
        "1": "",
        "2" : null
        },
    "obsIdsCell": {
    // "comp. id" : "field number of the absolute or gis id of cell attached to obs"
        "1" : "",
        "2" : null
    },
     "extNames": {
    // "compartment id" : "name of associated extraction shp in the QGIS project"
        "2": "",
        "1": ""
    },
    "extIdsColNames": {
    // "compartment id" : "field number containing the extraction point name"
        "2": "",
        "1": ""
    },
    "extIdsColLayers": {
    // "compartment id" : "field number containing the aquifer layer identifier"
        "2": null,
        "1": ""
    },
    "extIdsColCells": {
    // "compartment id" : "field number containing the cell identifier"
        "1": "",
        "2": "",
    }
}
}
```

Code 2.1: Contents of a geometry configuration file

2.3.4 Contents of the Post-Processing Directory

Upon execution of the CaWaQS-Viz application parameterisation dialog box, the project sub-directory is created in the specified post-processing directory. If the project does not yet exist in this directory, a sub-directory named after the project will be created, containing the project and resolution configuration files written in "`json`" format.

2.4 Project Configuration from a `.json` Project Configuration File

Importing an external `.json` file can also be used to configure the project. This file is structured as follows:

```
{
    "json_path_geometries": str,
    "projectName": str,
```

```
    "cawOutDirectory": str,
    "startSim": int,
    "endSim": int,
    "obsDirectory": str,
    "ppDirectory": str,
    "regime": str
}
```

The contents of each object in this `.json` file are as follows:

- "`json_path_geometries`": absolute path of the resolution configuration file;
- "`projectName`": project name;
- "`cawOutDirectory`": absolute path of the directory containing the CaWaQS modelling outputs;
- "`startSim`" and "`endSim`": simulation start and end years in `integer` format;
- "`obsDirectory`": absolute path of the parent directory containing the observed time series;
- "`ppDirectory`": absolute path of the post-processing directory. Note that the path must already exist. Otherwise, it will not be created by the application.
- "`regime`": modelling regime ("Transient" or "Steady")

CHAPTER 3

CaWaQS-Viz Features

Contents

All shapefile outputs generated by CaWaQS-Viz are temporary. If they are not saved, they will no longer be accessible when the project is reopened.

3.1 "Calibration Tools": Calibration Tools

The "**Calibration Tools**" dialog box (**??**) provides access to the CaWaQS model calibration tools. These features are as follows:

- a tool for modifying the parameterisation files of the various modelled compartments: "**Modify a parametrisation file**" (**??**) ;
- a tool for calculating model calibration performance criteria : "**Statistical Criteria**" (**??**) ;
- a tool for calculating simulated run-off coefficients: "**Run-off coefficient calculation**" (**??**) ;
- a tool for visualizing simulated and observed time series: "**Compare sim/obs**" (**??**).

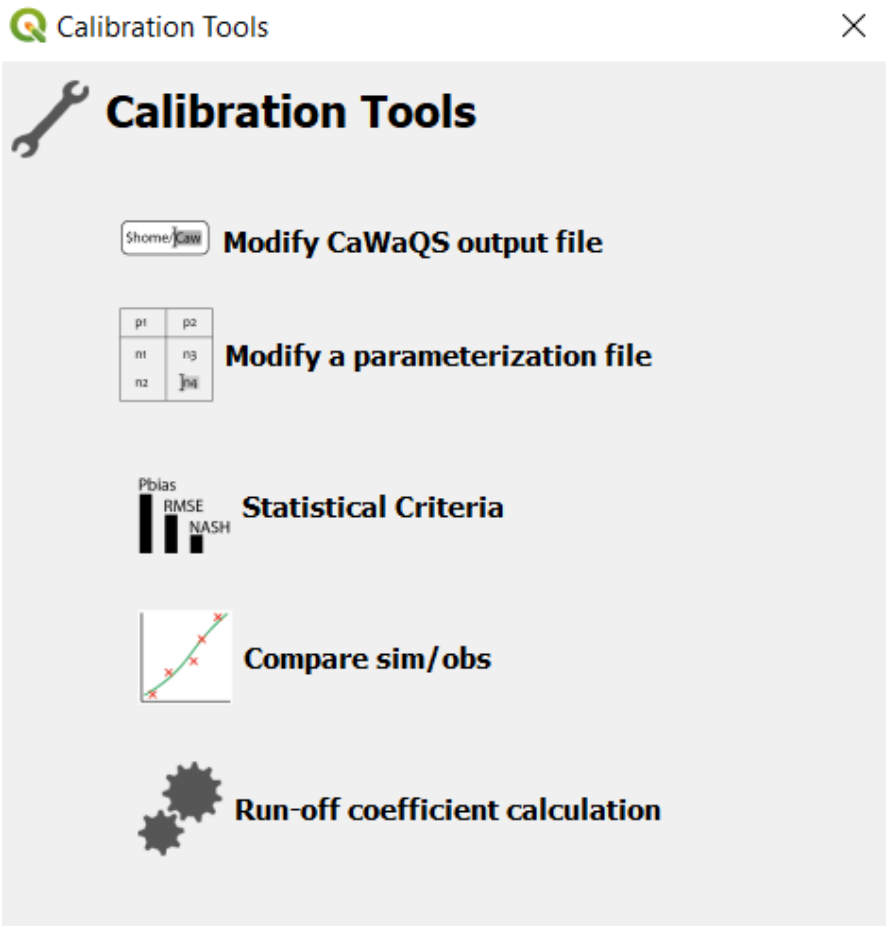
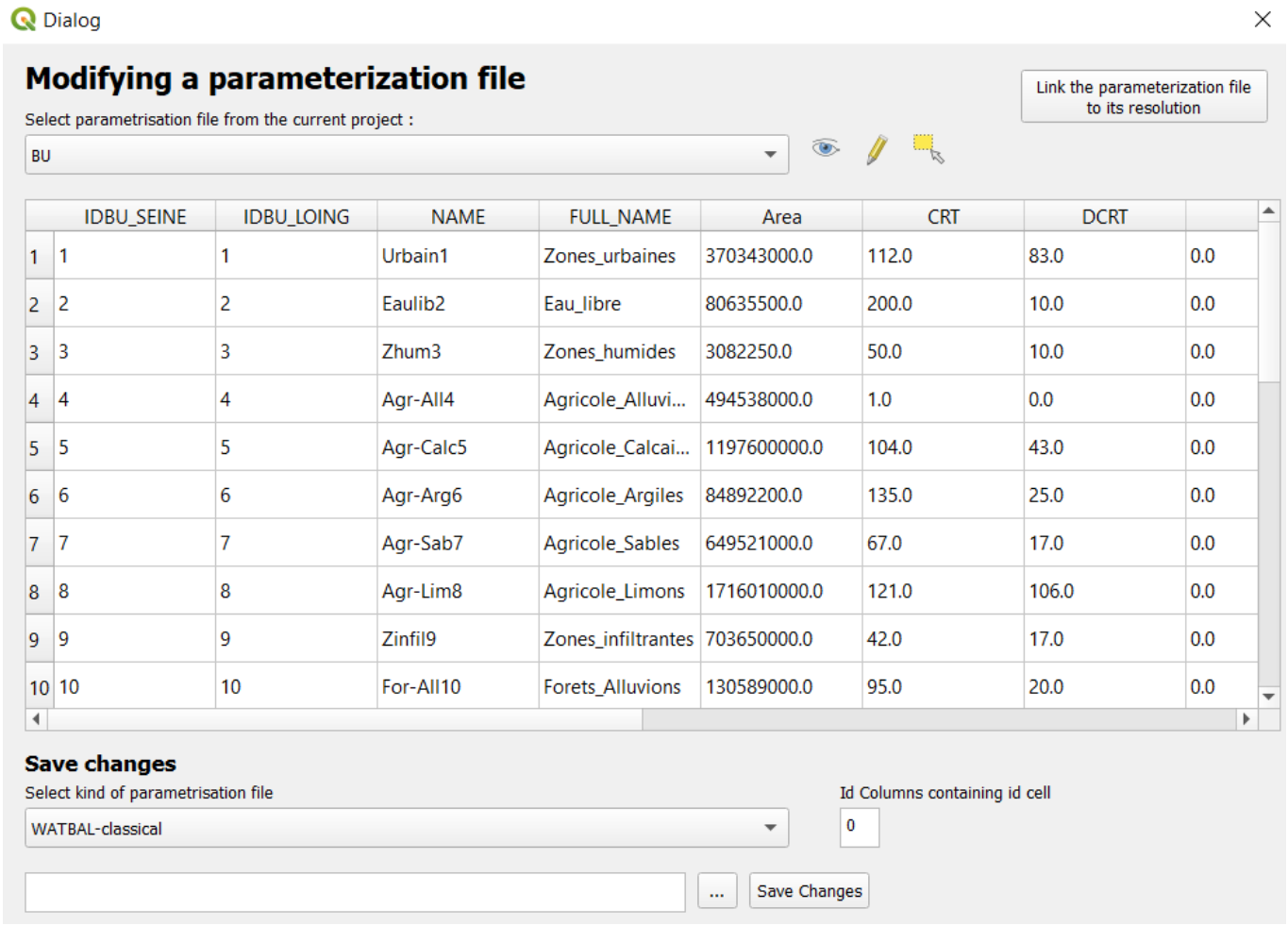


Figure 3.1: Calibration tools dialog box.

3.1.1 Modifying Parameterisation Files

Parameterisation files can be modified from the "`Modifying a parametrization file`" dialog box (**??**). The resolution will depend on the calibration file to be selected from the drop-down menu. The table can be displayed by clicking on .

If the displayed attribute table does not contain the fields of the parameterisation file, the resolution can be linked to its parameterisation file in the "`Link resolution to its parameterisation file`" dialog box, accessible by clicking on Link the parameterization file to its resolution . The use of this dialog box is described in **??**.



	IDBU_SEINE	IDBU_LOING	NAME	FULL_NAME	Area	CRT	DCRT	
1	1	1	Urbain1	Zones_urbaines	370343000.0	112.0	83.0	0.0
2	2	2	Eaulib2	Eau_libre	80635500.0	200.0	10.0	0.0
3	3	3	Zhum3	Zones_humides	3082250.0	50.0	10.0	0.0
4	4	4	Agr-All4	Agricole_Alluvi...	494538000.0	1.0	0.0	0.0
5	5	5	Agr-Calc5	Agricole_Calcai...	1197600000.0	104.0	43.0	0.0
6	6	6	Agr-Arg6	Agricole_Argiles	84892200.0	135.0	25.0	0.0
7	7	7	Agr-Sab7	Agricole_Sables	649521000.0	67.0	17.0	0.0
8	8	8	Agr-Lim8	Agricole_Limons	1716010000.0	121.0	106.0	0.0
9	9	9	Zinfil9	Zones_infiltrantes	703650000.0	42.0	17.0	0.0
10	10	10	For-All10	Forets_Alluvions	130589000.0	95.0	20.0	0.0

Figure 3.2: Dialog box for modifying parameterisation files.

Once the parameters have been modified in the displayed table, it can be saved. The new file path can be specified by selecting ... . Specify the identifier of the column containing the cell identifier or name[1].

[1]ATTENTION: As a reminder, column identifiers start at **0**.

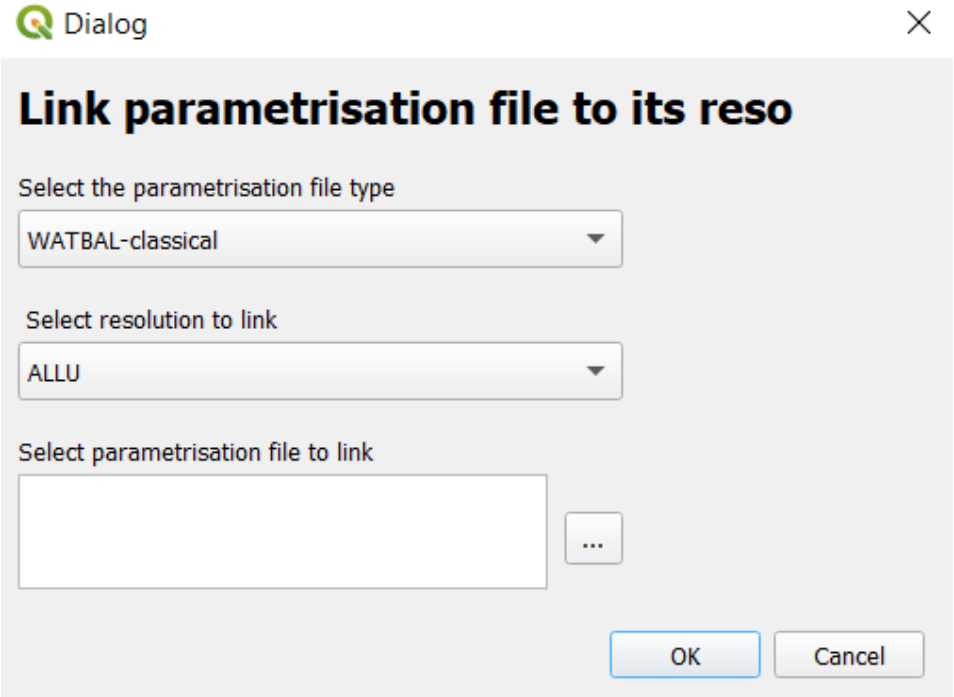


Figure 3.3: Dialog box for linking an attribute table to its resolution.

3.1.2 Linking an Attribute Table to its Resolution

The parameters from the parameterisation file will be directly imported as fields in the attribute table of the relevant resolution file. To create this link, a parameterisation file type must be chosen (based on the compartment and the type of parameterisation file), the CaWaQS resolution, and the path to this parameterisation file. Different types of parameterisation files exist and are listed in table **??**.

Compartment	Parameterisation file type	Parameters listed in the table
`WATBAL`	Classical	CRT, DCRT, R, FN, CQR, CQRMAX, CQI, CQIMAX and RNAP
`WATBAL`	Infiltration Excess	CRT, DCRT, R, FN, CQR, CQRMAX, CQI, CQIMAX, RNAP and RINF
`AQ`	Transmissivity	T
`AQ`	Storage (storage coefficient)	S
`AQ`	Conductance	C

Table 3.1: Parameterisation files supported by `CaWaQS-Viz`.

3.1.3 `Statistical Criteria`: Calculation of Statistical Performance Criteria at Observation Points

The "`Statistical Criteria`" dialog box (**??**) returns a GIS layer following the geometries of the observation resolution inherent to the analysed variable. Each field corresponds to a given statistical criterion. The selected statistical criteria are calculated over the period specified in the "`Period for calculating statistical criteria`" section.

3.1.3.1 Statistical criteria calculated by the "`Statistical criteria`" dialog box

"`Pbias`" : Bias [%]

$$PBIAS = \frac{\sum(\hat{y}_i - y_i)}{\sum y_i}.100$$

y_i : simulated variable at time i

O_i : observed variable at time i

"`Average SIM/OBS`"

$$A = \overline{\hat{y}} - \overline{y}$$

$\overline{\hat{y}}$: mean of simulated variables

$\overline{y}$: mean of observed variables

"Nash" : Nash-Sutcliffe Model Efficiency (?)

$$NSE = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2} \qquad \begin{array}{ll} y_i : & \text{actual variable at time } i \\ \hat{y}_i : & \text{predicted variable at time } i \end{array}$$

"RMSE" - Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \quad \text{where} \qquad \begin{array}{ll} n : & \text{Total number of simulation days} \\ y_i : & \text{actual variable at time } i \\ \hat{y}_i : & \text{predicted variable at time } i \end{array}$$

"KGE" - Kling-Gupta Efficiency (?)

$$KGE = 1 - \sqrt{(r-1)^2 + (\alpha - 1)^2 + (\beta - 1)^2}$$

r : Pearson correlation coefficient between the simulated and observed variable series

α : ratio of simulated and observed standard deviations

β : mean bias between simulated and observed series

(3.1)

The following criteria are calculated automatically: $\overline{y}$, $\overline{\hat{y}}$, $\frac{\overline{\hat{y}}}{\overline{y}}$, σy, $\sigma \hat{y}$, $\frac{\sigma \hat{y}}{\sigma y}$.

3.1.3.2 Output of the "Statistical Criteria" dialog box

The dialog box generates statistical criteria:

- at observation points,
- at the scale of the aquifer layer in the case of a simulation with an aquifer compartment;
- at the scale of the hydrosystem in the case of a simulation with an aquifer compartment.

Statistical criteria calculated at observation points

The dialog box generates the statistical criteria table at observation points in **shapefile** and **text** format. The Shapefile is introduced directly into the current **QGIS** project, while the text file will be found in the `../PostProcessing/STATS_CRITS` directory. The output criteria are contained in the fields of each measurement point under the names defined in table **??**.

Statistical criteria calculated at the scale of aquifer units

In the case of a calculation of statistical criteria on the aquifer compartment, a **text** file is saved in the "Post-Processing" directory under `../PostProcessing/STATS_CRITS/AQ_crits_xxxx_xxxx.txt`. This file lists the statistical criteria from table **??** defined at the scale of each aquifer unit.

Statistical criteria calculated at the hydrosystem scale

Statistical criteria are defined at the hydrosystem scale for the groundwater hydraulic head. These criteria are written in the file `../PostProcessing/STATS_CRITS/AQ_crits_xxxx_xxxx.txt`.

Output table field name	Statistical criterion
"NObs"	Number of observations over the selected period
"AObs"	Mean of observed variables
"ASIM"	Mean of simulated variables
"SUM_RATIO"	Ratio of the sum of simulated variables to the sum of observed variables
"StdObs"	Standard deviation of observed variables
"StdSim"	Standard deviation of simulated variables
"STD_RATION"	Ratio of standard deviations of observed and simulated variables
"PBAIS"	Bias [%]
"AVERAGE SIM/OBS"	Mean ratio between simulated and observed variables
"KGE"	Kling-Gupta Efficiency
"NASH"	Nash-Sutcliffe Model Efficiency
"RMSE"	Root Mean Squared Error

Table 3.2: Field names of output tables for each statistical criterion

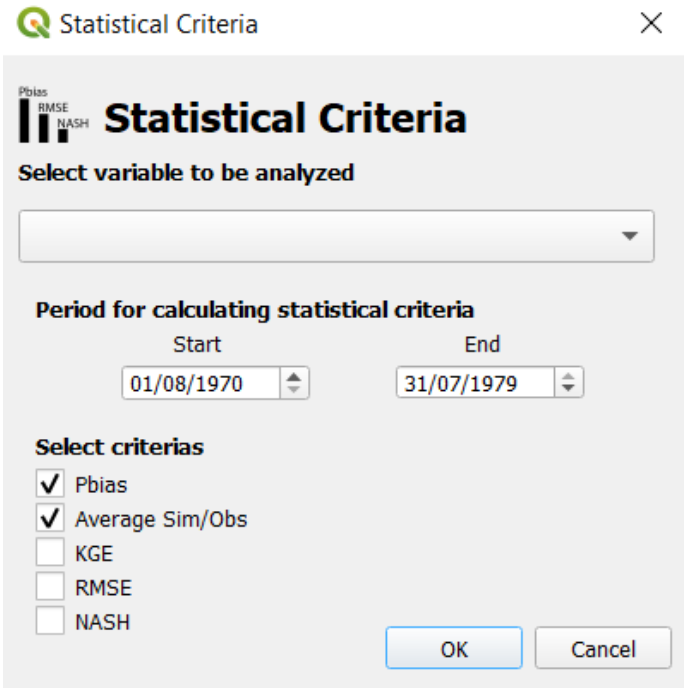


Figure 3.4: "Statistical Criteria" dialog box.

3.1.4 Run-off coefficient calculation: Calculation of the Run-off Coefficient at Flow Measurement Stations

This dialog box calculates the simulated and observed run-off coefficient at flow measurement stations.

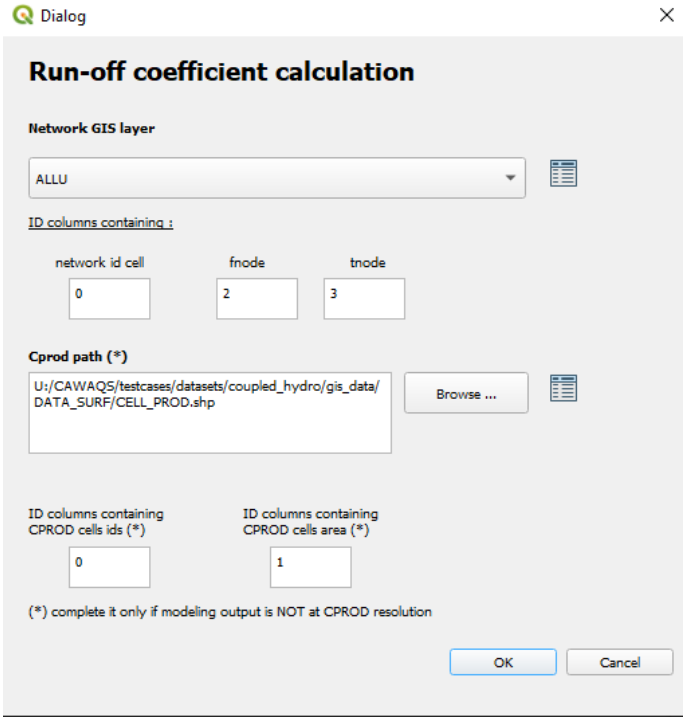


Figure 3.5: Dialog box for calculating the run-off coefficient.

This dialog takes as input the resolution of the surface hydrological scheme at the river reach scale and the CaWaQS surface resolution of the production cells (unit catchments or `Cprod`). For the former, the column number containing the reach identifiers, `Fnode` and `Tnode` must be specified. For the latter, the identifiers of the fields containing the production cell identifier, and the cell area.

If the surface resolution defined in the CaWaQS-Viz configuration is already that of production cells, it will not be necessary to fill in the lower part of the dialog box.

Two new resolutions (*shapefiles*) will be returned by the dialog box and added to the current GIS project:

- "`CATCH`" which contains the catchments drained by each measurement station;
- "`CATCH_SURF`", the product of the intersection between the surface resolution defined in the CaWaQS-Viz configuration and "`CATCH`". Its attribute table contains several fields:
  - "`CATCH_ID`" and "`CATCH_SURF`": the identifier of the catchment drained by the station and its area;
  - "`ID_OBS`": the identifier of the measurement station;
  - "`SURF_ID`": the area of the surface cell from the surface resolution defined in the CaWaQS-Viz configuration;
  - "`SURF_SURF`": its area;

- "INTER_ID": the identifier of the intersection cell;
- "INTER_SURF": the intersection area.

These resolutions are saved in temporary format. It is left to the user to save them for future reuse. If CaWaQS-Viz recognises these resolutions in the current QGIS project, they will be reused for calculating run-off coefficients.

These coefficients can be found in the attribute table of the flow observation layer specified at the configuration step. Two new fields are then added: "QruissSim" and "QruissObs" designating respectively the run-off coefficient simulated by CaWaQS and that observed at the measurement stations.

3.2 Compare sim/obs: Comparison of Observed and Simulated Time Series

This calculation module enables comparison between simulations and observations through several types of output:

- by clicking on "PDF printout", a pdf file is generated. The simulated and observed time series are presented with the statistical criteria listed in table **??**;
- by clicking on "Interactiv graphics", an html format reproduces the time series presentation format of the pdf interactively in the default browser;
- by clicking on "Statistical criteria", the "Statistical Criteria" dialog box (**??**) opens. The parameters specified in the "Compare Sim-Obs" dialog box (dates, compartment and observation unit of measurement) are transferred to the "Statistical Criteria" dialog box.

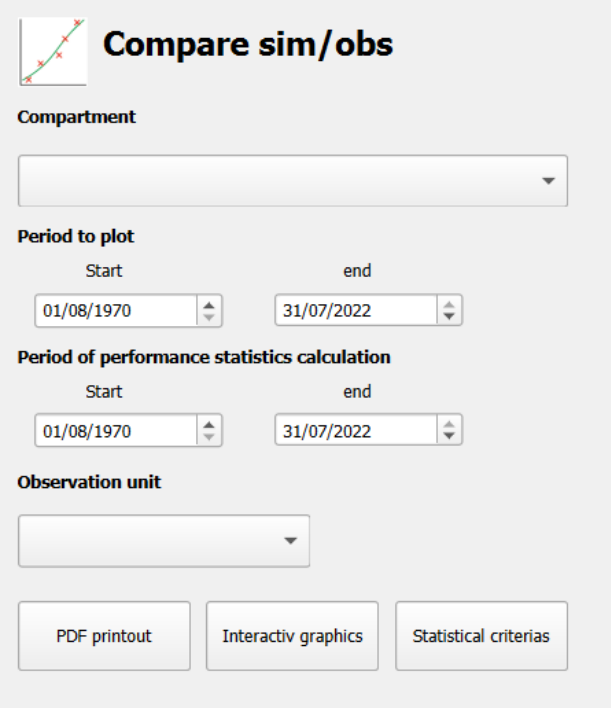


Figure 3.6: "Compare sim/obs" dialog window.

The compartment type (AQ = aquifer and HYD = hydrographic network) is to be selected from the drop-down list located in the upper part of the dialog box (??). The time series printing periods and those for calculating statistical criteria must also be specified. The unit of flow observations, used for processing simulations of the "HYD" compartment, must be specified in the "Observation unit" drop-down menu. It is optional for processing the "AQ" compartment.

An information message will be displayed in the QGIS interface at the end of this task and will point to the post-processing directory. The .pdf files resulting from analysis of the HYD compartment will be named "SIM_OBS_HYD.pdf" by default. Those from aquifer compartment analysis will be named "SIM_OBS_AQ.pdf". Each page of this document corresponds to a measurement station.

3.3 Spatial Map: Spatial Mapping of Hydrological Variables

This tool spatialises the surface water balance in .shp format. The balance can be produced at an annual, monthly or daily frequency (to be specified in the frequence field). Temporal aggregation can be performed by mean, sum, maximum or minimum (to be specified in the agregator field).

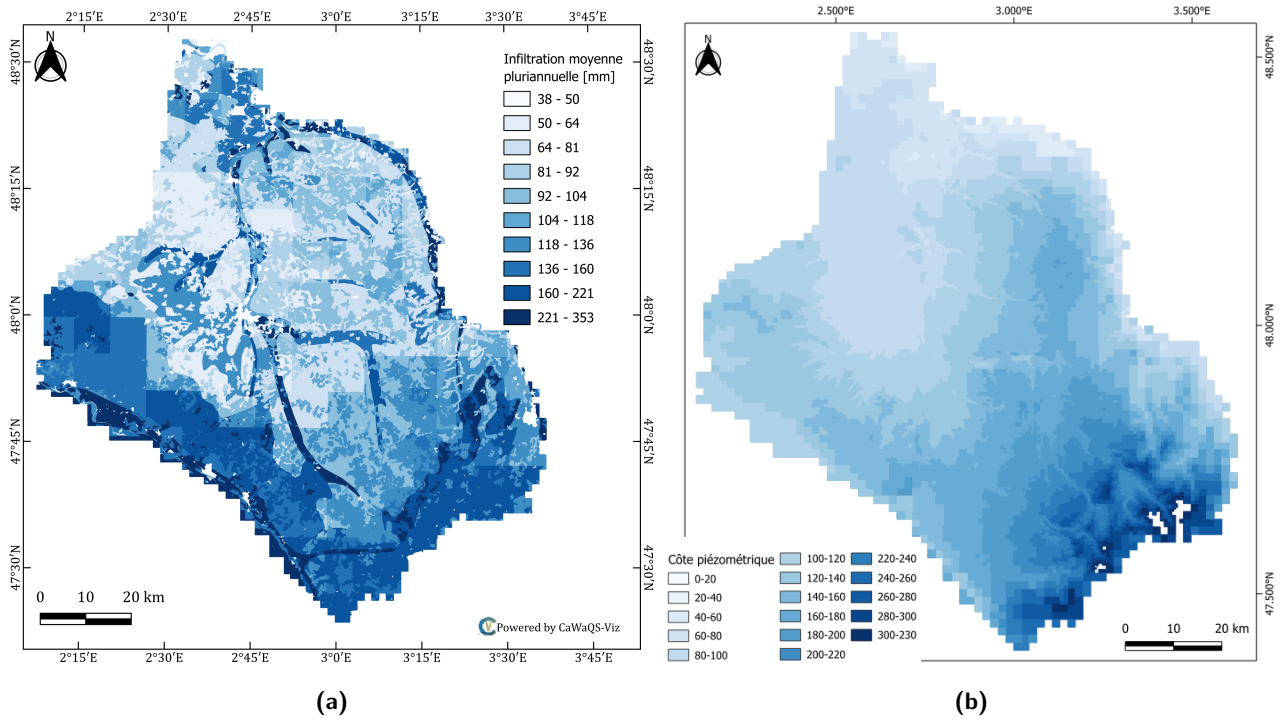


Figure 3.7: Spatialisation of mean inter-annual infiltration flux [mm] (a) and piezometry [$mNGF$] (b) simulated at the scale of the Loing hydrosystem

The values taken by the hydrological variable to be spatialised are contained in the attribute table of the output .shp resolution. The table has as columns the year identifiers for annual aggregation, the month identifiers for monthly aggregation and finally the date for daily time step output. A multi-year mean

aggregation is also possible by checking the `Pluriannual` box, thus establishing a multi-year or multi-year monthly balance.

- `WATBAL`: the precipitation, AET, runoff and infiltration variables (**??**);
- `AQ`: hydraulic heads (**??**), viewable either on a given aquifer mesh layer, or on the outcropping aquifer cells (unique mesh, created by CaWaQS-Viz).

Variables from the `WATBAL` compartment are expressed in $mm\,j^{-1}$ or $mm\,an^{-1}$ depending on the chosen aggregation mode. Variables from the `AQ` compartment are expressed in metres.

3.4 `Budget Bar Plot`: Hydrological Budget Visualization

Simulated hydrological budgets can be visualized from this dialog box. They are constructed in accordance with the specified period. The `Frequences` field specifies the water balance calculation time step (multi-year, annual, monthly or daily), while `agregator` specifies the temporal aggregator (mean, sum, maximum, minimum). The water balance is spatially aggregated by a mean of hydrological variables over all surface cells.

Upon execution of this dialog box, an `HTML` page opens in your browser and displays an interactive hydrological budget chart. A static image (cf. Fig. **??**) of this same chart is saved in the post-processing directory in a sub-folder `../BUDGET` of the post-processing directory.

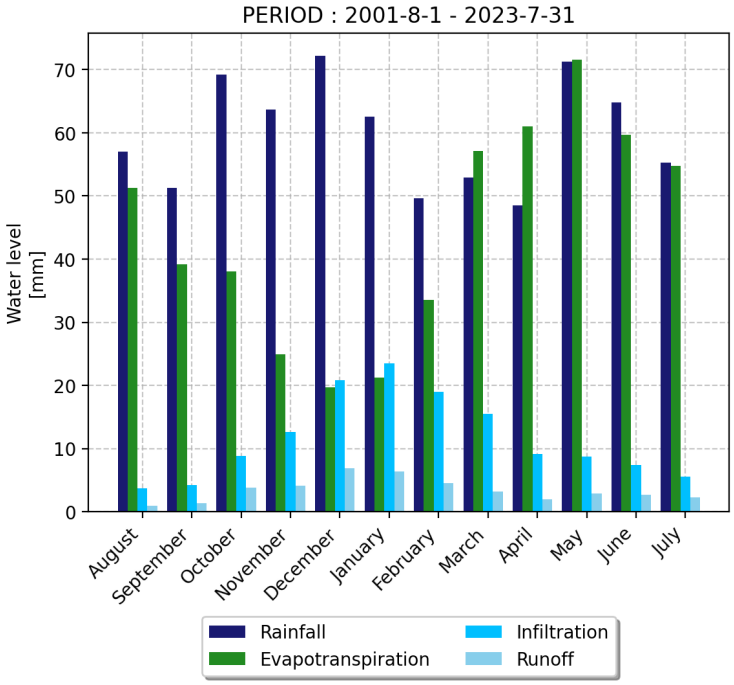


Figure 3.8: Example of a graphical output for the simulated multi-year monthly hydrological budget

3.5 `Hydological Regime`: Hydrological Regime Visualization

The simulated hydrological regimes of the hydrosystem are constructed from the "**`Hydrological Regime`**" dialog box (cf. Figure **??**). Hydrological regimes are calculated over the period specified in the "**`Start`**" and "**`end`**" input fields. The variable to be analysed is to be specified in the "**`Hydrological variable to plot`**" drop-down menu. The output type is chosen by the user by checking the "**`static plots (PNG)`**" box for graphical outputs as individual images for each measurement point, "**`static plots (PDF)`**" for a PDF output compiling the hydrological regimes of all measurement points, and finally "**`interractives plots`**" for an output in the default browser in the form of interactive charts (**??**). A dialog box alerts the user when the calculation module has finished executing.

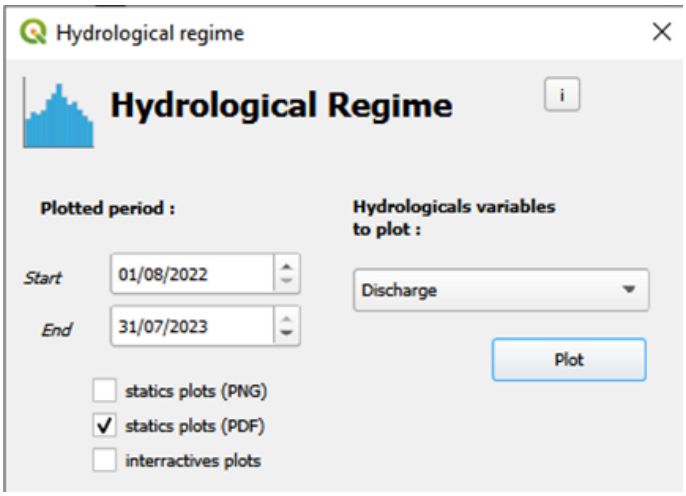


Figure 3.9: "**`Hydrological Regime`**" dialog box for calculating multi-year monthly hydrological regimes.

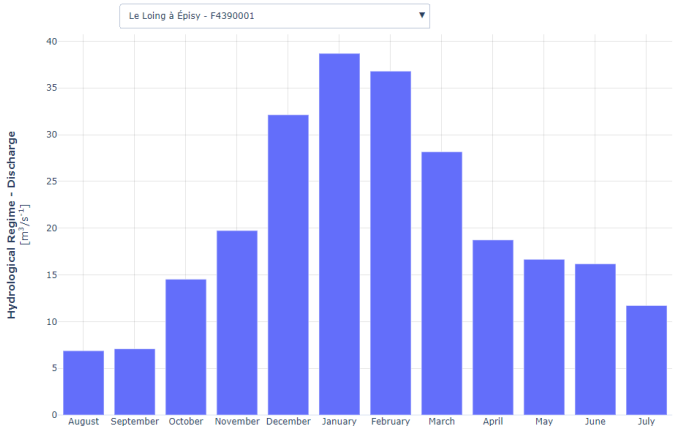


Figure 3.10: Example of a graphical output for the simulated multi-year monthly hydrological regime

3.6 `Extract Data from a sub area`: Extraction of Results in a Polygon

The "`Extract Data from a Sub Area`" dialog box allows extracting simulation results over a defined zone. This zone can be specified either as a polygon from a `QgsVectorLayer` loaded in the project, or as a shapefile imported from a directory.

Results are saved as CSV files (analyses) and NumPy files containing the data from the extracted zone. These files allow rerunning analyses from the created configuration.

3.6.1 Extraction Parameters

From the dialog box, the following parameters can be specified for extraction:

- Name of the new extraction file (also used as the name of the folder created in `PostProcessing/DATA_EXTRACTED/`)
- Extraction polygon: either a shapefile or a `QgsVectorLayer` present in the project;
- Extraction period;
- Extraction resolution for the WATBAL module.

3.6.2 Extraction Outputs

The outputs depend on the boxes checked in the right-hand column.

3.6.2.1 Outputs – Hydrological Balance (WATBAL)

The following variables can be extracted:

- "`Rainfall`";
- "`Real EvapoTranspiration (ET)`";
- "`Infiltration`";
- "`Runoff`";
- "`Potential EvapoTranspiration (ET)`".

These options provide the results of the hydrological balance (WATBAL). Data are saved in a CSV file in aggregated form over the entire simulation period, with one value per cell.

3.6.2.2 Outputs – Hydrographic Network (HYD)

"`Surface flow`"

- All reaches in contact with the polygon boundary are defined as extraction points;
- The CSV file contains daily flow and water level data for each point located on the polygon boundary;
- The sign convention for flow is: positive for inflow, negative for outflow;
- NumPy files are also generated, allowing further analysis of these data.

3.6.2.3 Outputs – Aquifer (AQ)

"`Subsurface flow`"

- For each aquifer layer, the polygon boundaries are defined with the resolution specific to that layer;
- A CSV file is generated for each layer;

- Fluxes are provided at a daily scale for each face located at the aquifer boundary;
- Faces are named according to the convention: `nCell(ID_ABS)_direction_unit`, for example: `3323_west_m3d`.

3.6.3 Output Organisation

For each extraction, result files and `QgsVectorLayers` are produced.

Files are saved in the output directory defined in the main analysis parameters: `PostProcessing/DATA_EXTRACTED/<E` containing the following sub-folders:

- **CONFIG/**: configuration files for rerunning simulations;
- **OUTPUT/**: CSV files containing the results;
- **TEMP/**: NumPy files used to rerun analyses.

3.6.4 Extraction GeoPackage Layers – Column Description

When running an area extraction from the `DialogExtract` interface, CaWaQS-Viz produces several GeoPackage (`.gpkg`) files loaded as `QgsVectorLayer` in the QGIS project. Each file describes either the *mesh* (cells or reaches inside the extracted sub-area) or the *boundary* (edges or points where fluxes cross the extraction polygon).

All GeoPackage files share one automatically generated column:

`fid` Auto-generated integer primary key, imposed by the GeoPackage format specification. It is a 1-based sequential index created by GDAL/OGR when GeoPandas writes the file; it carries no hydrological meaning.

3.6.4.1 Mesh Layers

Table 3.3: Columns common to all extraction mesh GeoPackages.

Column	Type	Description
`fid`	int	GeoPackage auto-generated primary key (1-based).
`ID_ABS`	int	1-based sequential identifier of the cell/reach within the extracted sub-area. For multilayer compartments (e.g. AQ), the numbering restarts at 1 for each layer.
`ORIG_CELL_ID`	int	Identifier linking back to the original GIS layer loaded in the QGIS project (see Table **??**).
`geometry`	geom	Polygon (WATBAL, AQ) or LineString (HYD).

Common columns across all mesh layers.

Origin of `ORIG_CELL_ID` per compartment.

Table 3.4: Mapping of `ORIG_CELL_ID` to the original GIS layer.

Compartment	GeoPackage	Source GIS layer	Source column
WATBAL	`mesh_wb_{name}.gpkg`	ELE_BU (e.g. `ELE_BU_DrainAGR`)	Column defined `ids_col_cell` `config_geometries.json` for compartment 3 (WATBA
HYD	`mesh_hyd_{name}.gpkg`	ELEMENTS_MUSKINGUM	Column defined `ids_col_cell` for com ment 2 (HYD). This is GIS identifier (`ID_GIS` as found in the MENTS_MUSKINGUM tribute table.
AQ	`mesh_aq_{name}_layer{n}.gpkg`	Aquifer grid (e.g. `AQ_GRID_TERT`)	Column defined `ids_col_cell` for com ment 1 (AQ).

Table 3.5: Compartment-specific columns in mesh GeoPackages.

Compartment	Column	Type	Description
AQ	`ID_LAYER`	int	Index of the aquifer layer (e.g. 0, 1, . . .). Identifies which geological layer the cell belongs to.

Additional columns per compartment. WATBAL and HYD mesh layers have no additional columns beyond the common ones.

3.6.4.2 Boundary Layers

HYD boundary — `hyd_boundary_{name}.gpkg` One Point feature per reach that crosses the extraction polygon boundary.

AQ boundary — `aq_boundary_{name}_layer{n}.gpkg` One LineString feature per boundary face (shared edge between an interior cell and an exterior neighbour).

3.6.4.3 Example: Extraction with Two Aquifer Layers

For an extraction named `MyArea` on a project with WATBAL, HYD, and two AQ layers (0 and 1), the following GeoPackages are produced:

1. `mesh_wb_MyArea.gpkg` — WATBAL extracted cells (Polygon)

Table 3.6: Columns of the HYD boundary GeoPackage.

Column	Type	Description
`fid`	int	GeoPackage auto-generated primary key.
`ID_ABS`	int	1-based sequential rank of the boundary reach, matching the compact numpy row index used internally. Same ranking space as in the HYD mesh GeoPackage.
`NAME`	str	Label `boundary_{rank}`, used as extraction point name in re-analysis dialogs.
`LAYER`	int	Always 0 (required by the `Extraction.defineExtPoints()` interface).
`SIGN`	int	Flow direction: $+1$ = inflow (entering the sub-area), -1 = outflow (leaving the sub-area).
`geometry`	geom	Point at the polygon–reach intersection.

2. `mesh_hyd_MyArea.gpkg` — HYD extracted reaches (LineString)
3. `hyd_boundary_MyArea.gpkg` — HYD boundary crossing points (Point)
4. `mesh_aq_MyArea_layer0.gpkg` — AQ layer 0 extracted cells (Polygon)
5. `aq_boundary_MyArea_layer0.gpkg` — AQ layer 0 boundary edges (LineString)
6. `mesh_aq_MyArea_layer1.gpkg` — AQ layer 1 extracted cells (Polygon)
7. `aq_boundary_MyArea_layer1.gpkg` — AQ layer 1 boundary edges (LineString)

Table 3.7: Columns of the AQ boundary GeoPackage.

Column	Type	Description
`fid`	int	GeoPackage auto-generated primary key.
`ORIG_CELL_ID`	int	CaWaQS internal cell identifier of the boundary cell (same as in the AQ mesh GeoPackage).
`ID_ABS`	int	1-based sequential rank of the cell within the extracted sub-area for this layer.
`DIRECTION`	str	Cardinal direction of the boundary face (e.g. `N`, `S`, `E`, `W`).
`ID_LAYER`	int	Index of the aquifer layer.
`geometry`	geom	LineString representing the shared edge.

CHAPTER 4

Code Description

The CaWaQS-Viz plug-in is a fully-fledged Python class ("`cawaqs_viz.py`"). This class manages in particular the plug-in loading functions in the QGIS interface and links to the interface dialog boxes. The CaWaQS-Viz plug-in is structured around four main classes (cf. Fig. **??**):

- "`Compartement`" and "`Manage`" (*backend*) are object management classes attached to the CaWaQS model structure. They bring together all the features for analysing simulation results;
- "`ExploreData`" (*gateway*) is a backend object management class that allows navigating through the objects initialised during use of the plug-in;
- "`Dialog`" (*frontend*) is the parent class of all the plug-in dialog boxes. Each simulation result calculation and analysis action is launched from these dialog boxes.

Additional utilities are contained in the "`./tools`" directory. They include in particular functions for managing and processing QGIS objects (functions for layer creation, attribute table modification, adding entities to an attribute table, table refresh, etc.), as well as the statistical criteria calculation functions called in the class methods of "`Manage`".

Dialog boxes are created and modified using the "`QT Designer`" tool, provided with the QGIS installation. All objects inherent to the dialog boxes are included *via* this application. "`QT Designer`" creates the extension `.ui` file called by the object's class. In general, this file manages the appearance of the dialog box and its graphical objects. Each dialog box is therefore a class that calls its `.ui` file when the class is initialised.

Analysis features are activated by a specific action button or simply upon execution of the dialog box *via* its "`OK`" execution button. Each action button is attached to a class method of its dialog box. These methods call the "`ExploreData`" class to navigate through initialised objects and the "`Manage`" class if data processing is required. Features inherent to the use of QGIS objects (e.g.: displaying cartographic renderings, layer refresh, etc.) are directly managed by the dialog box class methods, which may sometimes call the additional utilities ("`tools`") if actions are shared between multiple dialog boxes.

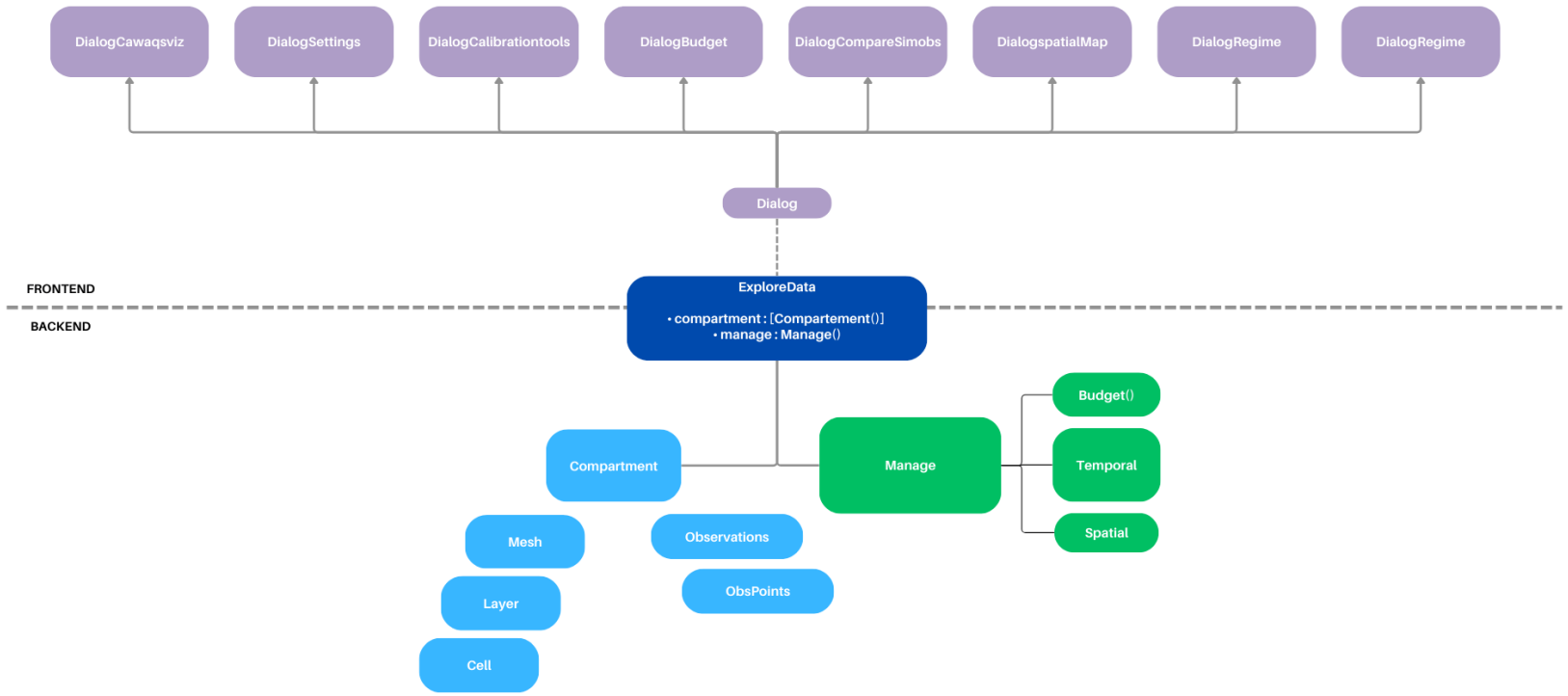


Figure 4.1: Source code structure of the CaWaQS-Viz plug-in.

CHAPTER 5

Developer Guidelines

5.1 Code Architecture

The code is organised around a backend and a frontend built around the dialog box classes. The backend embeds the simulation processing features (file reading, hydrological processing, etc.) as well as the mesh element classes of CaWaQS. "Gateway" classes for configuring the application from the QGIS instance and for navigating through simulation data and mesh elements link the backend and the frontend.

5.1.1 The backend

Embedding the classes (`Mesh`, `Layer`, `Cell`) constituting the CaWaQS mesh from `shapefile` layers present in the QGIS project, the backend also enables the hydrological processing and analysis of simulations, managed by the `Manage` class. The `Manage` processing class handles temporal (`Temporal`) and spatial `Spatial` processing of simulations as well as the calculation of hydrological and hydrogeological budgets (`Budget`). The `Compartment` class catalogues the mesh elements built with `Mesh`, which itself calls the `Mesh` class that builds the compartment layers (one layer per compartment except for the aquifer compartment). `Compartiment` also integrates the simulation directory paths for the hydrological variables simulated for that compartment and the observed variables directory. It also embeds as an attribute the `Observation` class integrating `ObservationPoints` when they are specified, and the `Extraction` class integrating `ExtractionPoints` when they exist.
Finally, the `parameters.py` file integrates all the parameterisation variables of the `Manage` class inherent to the CaWaQS writing structure, namely the fields written in binary files, the writing formats as well as the CaWaQS code structure, namely compartment names, their index, the names of the output sub-directories written by CaWaQS during simulation, the structures of the `corresp_file` files written by the CaWaQS platform during simulation for each compartment whose processes are simulated. The distinction of these variables allows modifying application parameters without modifying all variables across the different code classes.

5.1.2 The frontend

The frontend consists of the application dialog box classes associated with their `.ui` files that define the `Qt` objects present in the dialog box (fields, lists, text, etc.). These dialog boxes are created from the `Qt Designer`[1] application, embedded with QGIS. Section **??** explains how to use the dialog box design application and how it integrates into the CaWaQS-Viz application. The application frontend directly manages calls to the backend functions and processing of the data managed by `ExploreData` according to user requests. A process can be executed directly upon validation of the dialog box via the `ok` button, upon closing the dialog box, or via another command button in the dialog box (example: the `SpatialMap` dialog box contains as many buttons as there are hydrological variables that can be spatialised by the `CaWaQS` application).

5.1.3 The gateways

The gateway classes `ExploreData` and `Config` are, for the former, a class for navigating through the simulation and mesh data of a `CaWaQS` simulation, and for the latter, a class for configuring the `CaWaQS` application from a QGIS project configuration. The configuration class writes in `json` format the QGIS project geometric configuration (`config_geometrie.json`) and the simulation project configuration (`config_projet`) embedding the information necessary for processing `CaWaQS` simulations, namely the simulation periods, the simulation output directory, the observation directories when they exist, the QGIS project geometric configuration, the simulation regime and the compartments whose hydrological processes are simulated. This class then configures the `ExploreData` class, which calls the application backend by building the simulation compartments.

[1] downloadable from https://build-system.fman.io/qt-designer-download

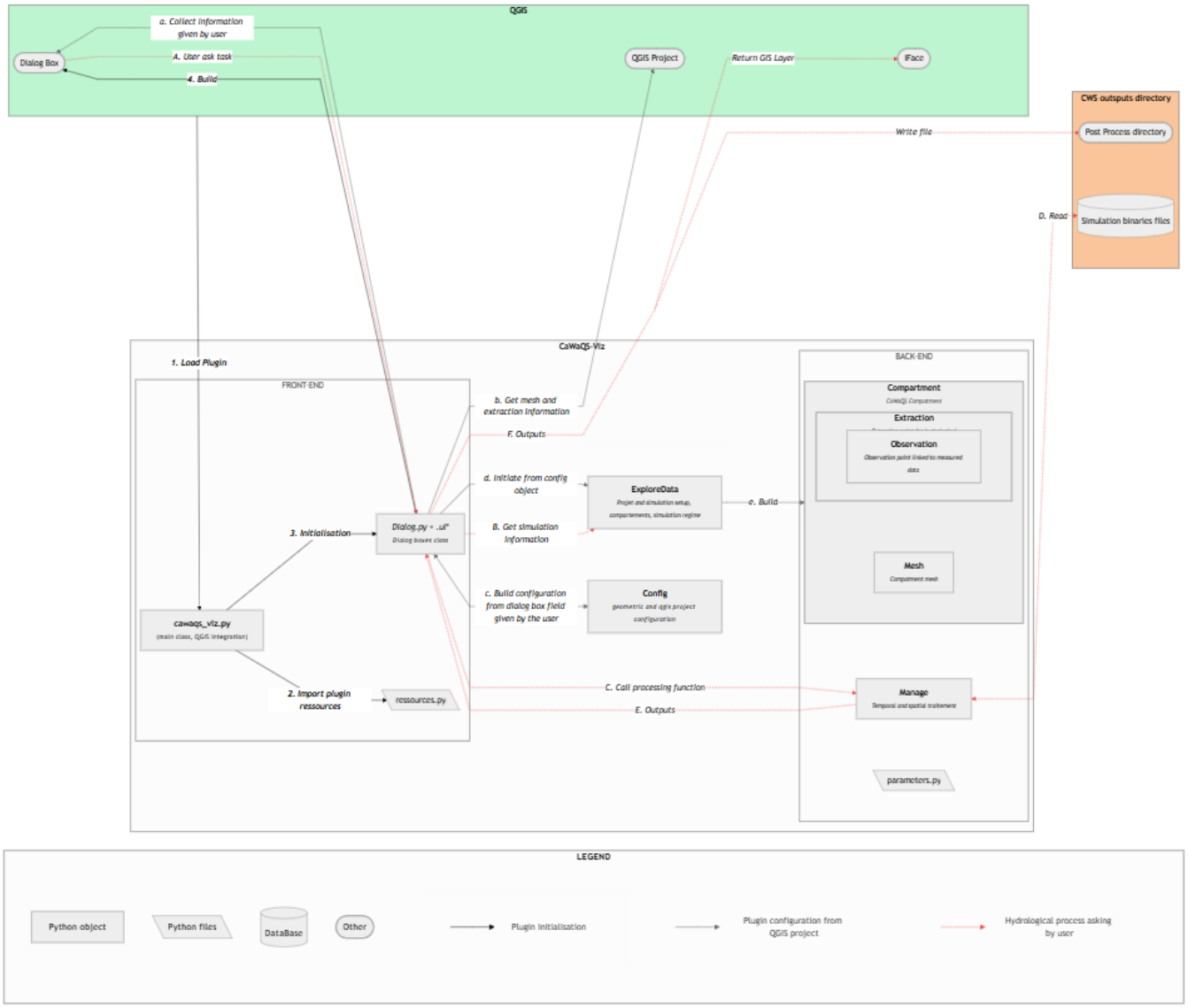


Figure 5.1: Functional diagram of the CaWaQS-Viz application

5.2 Version Update

The plugin version is specified in the `metadata.txt` file contained in code excerpt **??**.

```
[general]
name=CaWaQS-Viz
qgisMinimumVersion=3.0
description=Description
version=0.1.16
author=Lise-Marie GIROD (Mines Paris - PSL), Nicolas Flipo (Mines Paris - PSL)
email=lise-marie.girod@minesparis.psl.eu
```

```
8
9  about=Python QGIS-based vizualization software for CaWaQS3.x
10
11 repository=https://gitlab.com/gviz/cawaqsviz
```

Code 5.1: Mandatory information to fill in the `metadata.txt` file

5.3 Code Documentation

The code documentation is published on the following link via GitLab Pages: https://cawaqsviz-gviz-adf347ff6d9d45c174db091497a2dfc0f14305cdb4ed9070.gitlab.io/. The documentation is maintained through docstrings in the application modules, following the formatting shown in the example below.

```
1  class Exemple:
2      """
3      Resume des fonctionnalites de la classe.
4      """
5
6      def __init__(self, arg1):
7          """
8          Methode constructeur.
9
10         :param arg1: descriptif du parametre
11         :type arg1: str
12         """
13         self.arg1 = arg1
14
15     def do_something(self, arg1: list) -> str:
16         """
17         Descriptif de la fonction.
18
19         :param arg1: descriptif du parametre
20         :type arg1: list
21         :return: descriptif de ce que la fonction retourne
22         :rtype: str
23         """
24         return ", ".join(map(str, arg1))
```

Code 5.2: Docstring formatting for automatic integration into code documentation

Compilation is performed by `Sphinx`[2]. The `docs/sources` directory contains the configuration files (`Conf.py`), as well as the documentation content. When integrating new modules into the plugin, adding them to the index (`index.rst`) is necessary for the code docstring to appear on the documentation page. This index integrates 3 files: `frontend.rst`, `interface.rst`, `backend.rst`, which contain the classes to be documented. Integrating a new module into the application requires it to be added to these files in the format of code

[2]whose documentation is accessible via https://www.sphinx-doc.org/en/master/

excerpt **??**. Updates to the documentation source files must be committed and synchronised with the repositories.

```
Module
------

.. automodule:: cawaqsviz.Module
   :members:
   :show-inheritance:
   :undoc-members:
```

Code 5.3: Docstring formatting for automatic integration into code documentation

Finally, with each new commit to the `Main` or `Beta` branches, the documentation is compiled according to the content of the `docs/source` directory of the repository.

```
image: python:3.9

stages:
  - build
  - deploy

before_script:
  - pip install -U pip
  - pip install -r requirements.txt

build-docs:
  stage: build
  script:
    - sphinx-build -b html docs/source public
  artifacts:
    paths:
      - public

pages:
  stage: deploy
  script:
    - echo "Pages deployed"
  artifacts:
    paths:
      - public
  only:
    - main
    - beta
```

5.4 Debugging Tools

5.4.1 The `Plugin Reloader` plugin

This utility allows reloading a plugin after a code modification. Thus it is not necessary to reload QGIS after each code change.

The plugin can be installed from the `QGIS` extension manager.

Figure 5.2: Description sheet for `Plugin Reloader`

5.4.2 The QGIS plugin `First Aid`

The `First Aid` plugin allows investigating errors occurring during plugin execution.

5.4.3 VS-Code

The **debugvs** plugin allows connecting a **VS-Code** debugging instance to the plugin execution from QGIS. This debugging configuration allows using all VS Code features during debugging (breakpoints, watch, call stack...). Setting up this debugging configuration is done as follows:

1. install **debugpy** with the command **!pip install debugpy** from the QGIS Python command prompt;
2. if the plugin development directory is not located in C://Users/*USER*/AppData//QGIS/QGIS3/profiles/defaul create a symbolic link in this directory pointing to the development directory.
3. install the **debugvs** plugin by downloading its script from the GitLab repository https://github.com/lmotta/debug_vs and activate the plugin by importing the **.zip** folder in **QGIS** (cf. Fig. **??**);

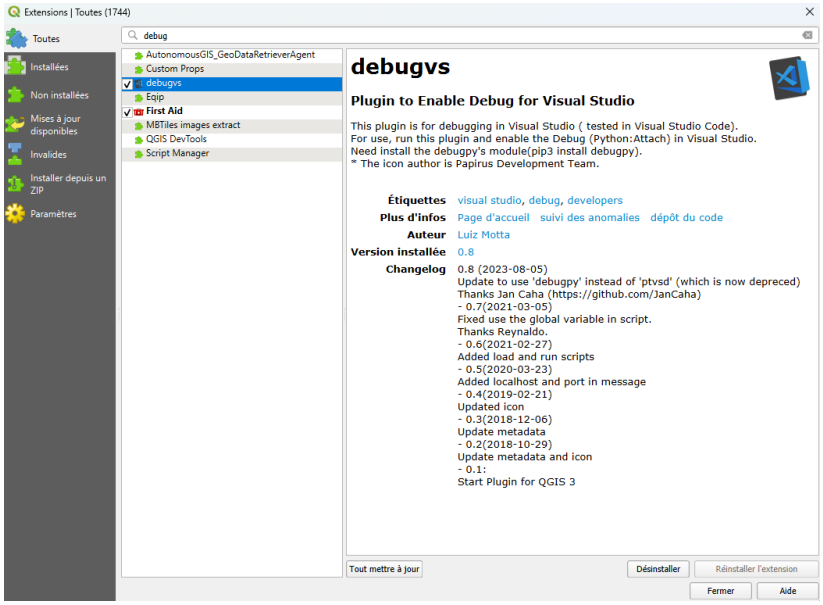


Figure 5.3: debugvs plugin

4. configure VS Code to connect to QGIS during a debugging session:
   (a) in the CaWaQS-Viz development directory, create a **.vscode** sub-directory (add this directory to the **.gitignore** file);
   (b) create a **settings.json** file in the **.vscode** sub-directory and copy the content from code **??**. Optionally modify the directory paths to match your QGIS and Python installation and operating system. This one is for Windows.

```
{

    "workbench.colorCustomizations": {
        "titleBar.activeBackground": "#006823",
        "editor.background": "#000000"
```

```
        },

        "python.linting.pylintEnabled": true,
        "python.jediEnabled": true,


        "python.pythonPath": "C:/OSGeo4W64/bin/python-qgis.bat",


        "python.linting.pylintArgs": [
            "--extension-pkg-whitelist=PyQt5,qgis,db_manager",
            "--disable=all",
            "--enable=F,E,unreachable,duplicate-key,unnecessary-semicolon,
global-variable-not-assigned,unused-variable,binary-op-exception,bad-format-
string,anomalous-backslash-in-string,bad-open-mode"
        ],


        "terminal.integrated.env.windows": {

            "PATH": "C:\\OSGeo4W\\apps\\qgis\\bin;C:\\OSGeo4W\\apps\\
Python312;C:\\OSGeo4W\\apps\\Python312\\Scripts;C:\\OSGeo4W\\apps\\Qt5\\bin;C
:\\OSGeo4W\\apps\\Python312\\Scripts;C:\\OSGeo4W\\bin;C:\\Windows\\System32;C
:\\Windows;C:\\Windows\\system32\\WBem;C:\\OSGeo4W\\apps\\Python312\\lib\\
site-packages\\pywin32_system32;C:\\OSGeo4W\\apps\\Python312\\lib\\site-
packages\\numpy\\.libs",

            "PYTHONHOME": "C:\\OSGeo4W\\apps\\Python312",
            "PYTHONPATH": "C:\\OSGeo4W\\apps\\qgis\\python;%PYTHONPATH%",

            "GDAL_DATA": "C:\\OSGeo4W\\share\\gdal",
            "GDAL_DRIVER_PATH": "C:\\OSGeo4W\\bin\\gdalplugins",
            "GDAL_FILENAME_IS_UTF8": "YES",

            "GEOTIFF_CSV": "C:\\OSGeo4W\\share\\epsg_csv",

            "O4W_QT_BINARIES": "C:/OSGeo4W/apps/Qt5/bin",
            "O4W_QT_DOC": "C:/OSGeo4W/apps/Qt5/doc",
            "O4W_QT_HEADERS": "C:/OSGeo4W/apps/Qt5/include",
            "O4W_QT_LIBRARIES": "C:/OSGeo4W/apps/Qt5/lib",
            "O4W_QT_PLUGINS": "C:/OSGeo4W/apps/Qt5/plugins",
            "O4W_QT_PREFIX": "C:/OSGeo4W/apps/Qt5",
            "O4W_QT_TRANSLATIONS": "C:/OSGeo4W/apps/Qt5/translations",
            "QT_PLUGIN_PATH": "C:\\OSGeo4W\\apps\\qgis\\qtplugins;C:\\OSGeo4W
\\apps\\Qt5\\plugins",

            "QGIS_PREFIX_PATH": "C:/OSGeo4W/apps/qgis",

            "VSI_CACHE": "TRUE",
            "VSI_CACHE_SIZE": "1000000"
        },
```

```
            "python.languageServer": "Jedi",
        }
```

Code 5.4: Workspace configuration file. This file defines the Visual Studio Code workspace configuration for QGIS plugin development. It adapts the VS Code Python environment to that used by QGIS (installed via OSGeo4W).

(c) create a `launch.json` file in the `.vscode` sub-directory, copy the content from code **??**, modify the `localRoot` and `remoteRoot` paths to match your workspace configuration.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Attach QGIS Debug",
            "type": "python",
            "request": "attach",
            "connect": {
                "host": "localhost",
                "port": 5678
            },
            "pathMappings": [
                {
                    "localRoot": "C:\\Program Files\\$QGIS$\\apps\\qgis-ltr\\
    python\\plugins\\cawaqsviz",
                    "remoteRoot": "C:\\Users\\$USER$\\AppData\\Roaming\\QGIS\\
    QGIS3\\profiles\\default\\python\\cawaqsviz"
                }
            ],
            "justMyCode": false
        }
    ]
}
```

Code 5.5: VS-Code debugger configuration file. In this configuration, the "connect" section indicates the address and port on which QGIS exposes the debugging session (here localhost:5678). The "pathMappings" block establishes the correspondence between the local project paths open in VS Code (localRoot) and the paths used by QGIS to execute the plugin (remoteRoot). This correspondence is essential for breakpoints placed in VS Code to be correctly associated with the files actually executed by QGIS.

5. Establish the QGIS/VS-Code connection

(a) authorise the debugger connection to QGIS via the `debugvs` plugin (cf. Fig. **??**);

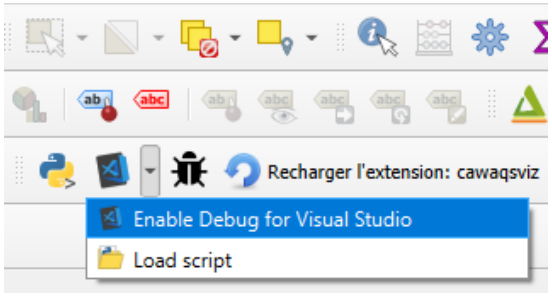


Figure 5.4: Establishing a QGIS/VS-Code connection from the `debugvs` plugin

(b) launch the VS-Code debugging session (cf. Fig. **??**).

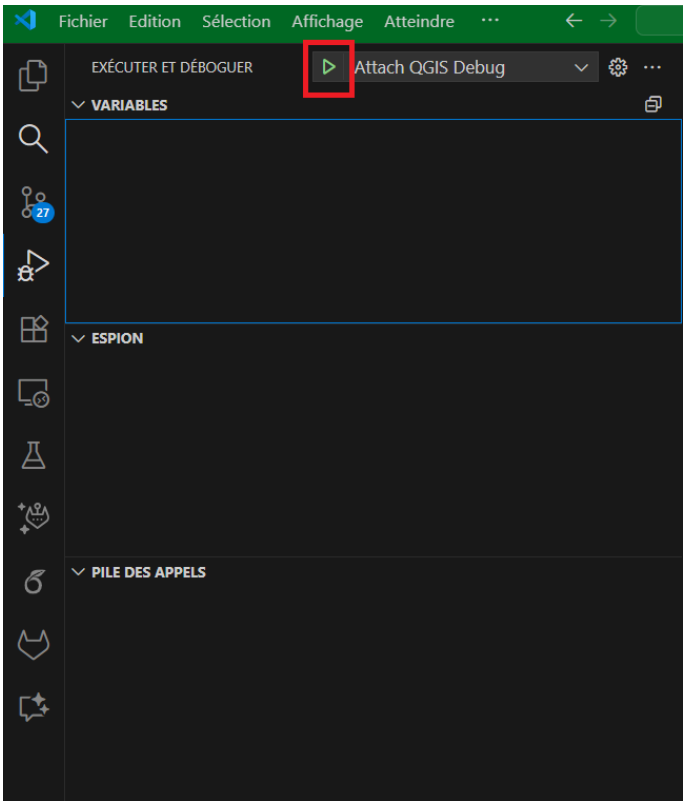


Figure 5.5: Launching a debugging session from VS-Code

The connection between the debugger and QGIS is terminated from VS-Code by clicking on `"Disconnected"` in the debugging toolbar (cf. Fig. **??**)

Figure 5.6: VS-Code debugger toolbar

5.5 Creating Dialog Boxes with Qt Designer

?? The **Qt Designer** application, provided with the QGIS installation (installation with OSGEO), is a tool for creating dialog boxes in the form of .ui files. In the CaWaQS-Viz application, all dialog boxes have their own **.py** file to which the **.ui** file, of the same name, is associated with the dialog box's Python class.

```
# This loads your .ui file so that PyQt can populate your plugin with the elements from Qt
    Designer
FORM_CLASS, _ = uic.loadUiType(
    os.path.join(os.path.dirname(__file__), "DialogComparesimobs.ui")
)


class DialogComparesimobs(Dialog, FORM_CLASS):
    def __init__(self, iface:QgisInterface, log_file_path:str, parent=None):
        """Constructor Method

        :param iface: Qgis Interface
        :type iface: QgisInterface
        :param logFilePath: Path of the cawaqs-viz log file, defaults to None
        :type logFilePath: str, optional
        :param parent: Parent class, defaults to None
        :type parent: _type_, optional
        """
        super(DialogComparesimobs, self).__init__(parent)
        self.setupUi(self)
        self.iface = iface
        self.logFilePath = log_file_path
        self.simData = False
```

Code 5.6: Initialisation of the "Compare SIM/OBS" dialog box from a .ui file